

1-5 · Docker Compose 이해하고 사용하기 · 실습 1.5H

다중 컨테이너 구성 실습

하나 → 둘 → 셋, 통합 환경을 한 번에 기동

수도권ICT 이노베이션스퀘어
데이터 엔지니어링 & RAG 파이프라인 160H · 3기

1.5시간 뒤에는

- ① 서비스 하나짜리 Compose 로 up·ps·logs·down 한 바퀴를 돈다
- ② 컨테이너 안에서 서비스 이름은 되고 localhost 는 실패함을 본다
- ③ db·admin·app 3개 서비스의 yml 을 쓰고 config 로 검사한다
- ④ up -d --build 로 한 번에 기동하고 psql·pgAdmin 으로 연결을 확인한다

일부러 실패시키는 단계가 하나 있습니다 — localhost 가 왜 안 되는지 이유가 남습니다

다섯 단계 — 작게 시작해 통합 환경까지

각 단계 끝에 확인 포인트가 있습니다. 통과한 뒤에 다음으로 넘어가세요

1 최소 Compose

nginx 하나 · up → ps
→ curl → logs →
down

2 둘로 늘리기

client 에서 web 을
이름으로 호출

3 통합 yml 작성

db · admin · app ·
docker compose
config

4 한 번에 기동

up -d --build · ps -a ·
logs app

5 연결 확인

psql · getent hosts
db · pgAdmin

오늘 쓰는 docker compose 명령 일곱 개

기동	docker compose up -d
상태	docker compose ps (-a 는 종료된 것까지)
로그	docker compose logs 서비스명
안에서 실행	docker compose exec 서비스명 명령
문법 검사	docker compose config
빌드 + 기동	docker compose up -d --build
정리	docker compose down

docker-compose.yml 이 있는 폴더에서 친다 — 다른 곳이면 「no configuration file provided」

1단계

가장 작은 Compose 파일부터 — 한 바퀴 돌아 보기

1 compose-test 폴더에 yml 하나
services 아래 web: image nginx:1.27 · ports "8080:80"

2 up -d → ps → curl → logs web → down → ps -a
끝의 compose ps -a 가 제목 줄만이면 정리 끝

3 확인 포인트
ps 에 web 이 running · curl 에 Welcome to nginx! · down 뒤 compose ps -a 는 제목 줄만

어디서 치나

yaml 있는
폴더에서만

| mkdir -p ~/data-eng/week1/compose-test 에서 시작합니다

client 컨테이너에서 web 을 두 가지 주소로 불러 본다

http://web:80 — 성공

서비스 이름으로 부른다 — 같은 네트워크, 이름이 곧 주소

- `exec client curl -s http://web:80`
- nginx HTML 이 돌아온다
- client 는 sleep infinity 로 띄워 둔다

VS

http://localhost:80 — 실패

같은 자리에서 localhost 로 부르면 자기 자신을 가리킨다

- `curl --max-time 3 http://localhost:80`
- 실패해야 성공 — 정상입니다
- 이론의 규칙을 직접 확인한 것

통합 환경 docker-compose.yml — 서비스 셋과 볼륨 하나

서비스	무엇으로	핵심 설정
db	postgres:16	5432:5432 · pgdata 볼륨
admin	dpage/pgadmin4:9.18	8081:80 · depends_on db
app	build: ./app	depends_on db · 1-4 앱
volumes	pgdata:	이름 볼륨 선언

db 환경변수 engineer · engineer123 · pipeline / admin 로그인 admin@example.com · admin123

config 로 해석만 먼저 — ls app 에 Dockerfile 이 있어야 build 된다. pgdata 는 실제로 week1_pgdata(폴더 이름이 붙는다) — 3차시 pgdata 와 다른 볼륨

한 번에 기동하고 서비스 간 연결을 확인한다

1 docker stop pg
3차시 pg 컨테이너가 5432 를 쓰고 있으면 먼저 정지

2 docker compose up -d --build
빌드와 기동을 한 번에 — ps -a 에 db·admin 은 running, app 은 exited (0) 이 정상(일회성 앱)

3 exec db psql ... SELECT current_database();
pipeline 이 나오면 DB 정상

4 exec admin sh -c "getent hosts db"
IP 주소와 db 가 함께 보이면 이름 해석 성공

logs app 에

**1-4 앱의
출력 문구**

pgAdmin 서버 등록 — 호스트는 반드시 db

1 **http://localhost:8081**
admin@example.com / admin123 으로 로그인

2 **Connection 탭 Host name/address = db**
localhost 를 넣으면 연결되지 않는다 — 컨테이너 안의 localhost 는 pgAdmin 자신

3 **engineer / engineer123 / pipeline**
왼쪽 트리에 pipeline 데이터베이스가 보이면 성공

다섯 단계를 통과했는지

- 1 ps 에 web running · curl 에 Welcome to nginx! · down 뒤 compose ps -a 는 제목 줄만
- 2 http://web:80 은 HTML, http://localhost:80 은 실패
- 3 docker compose config 가 오류 없이 해석 결과를 출력
- 4 db-admin running · app exited (0) · logs app 에 앱 출력
- 5 psql 에 pipeline · getent 에 IP 와 db · pgAdmin 트리에 pipeline

1-5 · 실습 1.5H 마무리

다음은 주차 산출물 실습 (1H)

down 뒤 up 을 해도 데이터가 남는지 확인하고 README 를 붙여 1주차 산출물을 완성합니다

✓ week1 표를 만들고 down → up 뒤 살아 있는지

✓ README.md — 남이 보고 재현할 수 있게

✓ down -v 로 완전히 내리고 처음부터 검증

✓ 산출물 제출 페이지 + 갤러리 공유

지금 상태

db·admin·app 이 떠 있는 채로 이어갑니다